

LATENT

덮인 나사에는 아직 색이 없습니다.

장르 3D 해체 퍼즐 플랫폼 웹 (모바일·데스크톱) 참가 개인 문서 게임 소개 및 설명

철판이 여러 겹으로 볼트 고정된 벽을 나사를 하나씩 풀어 해체하는 퍼즐입니다. 풀 나사는 아래 **홀더 3칸** 중 하나로 들어가고, 홀더는 처음 들어온 색으로 잠긴 뒤 **같은 색 3개**가 모이면 비워집니다. 판의 나사를 모두 빼면 그 판이 떨어져 나가고, 아래에 덮여 있던 것이 드러납니다.

다른 점은 하나뿐입니다. **레벨이 미리 존재하지 않습니다**. 판에 덮인 나사에는 색이 없습니다 — 숨겨 둔 것이 아니라 *아직 결정되지 않은* 것이고, 위의 판이 떨어져 손이 닿는 순간 AI가 **당신의 그 시점 상황을 보고** 색을 정합니다.

플레이 링크

플레이	https://taragamesdev.github.io/latent-nan2026/
소스 코드	https://github.com/TaraGamesDev/latent-nan2026
플레이 영상	제출 시 기입

링크를 클릭하면 브라우저에서 바로 실행됩니다. 설치·로그인·계정·결제가 필요 없고, 유료 API나 별도 라이선스도 사용하지 않습니다. 최초 로드 이후에는 네트워크가 끊겨도 정상 동작합니다.

게임 방법

목표

벽의 나사를 전부 빼내면 그 벽을 클리어하고 다음 벽으로 넘어갑니다. 벽은 판이 넓어지는 쪽보다 **겹이 깊어지는** 쪽으로 자라고, 홀더가 막히면 런이 끝납니다. 얼마나 멀리 가느냐가 기록입니다.

조작

나사를 한 번 누르는 것이 전부입니다. 모바일에서는 탭, 데스크톱에서는 클릭. 드래그도, 길게 누르기도, 제스처도 없습니다.

규칙

상황	결과
풀기	위에 다른 판이 덮이지 않은 나사만 누를 수 있습니다. 누르면 돌아 나와 같은 색 홀더로 들어갑니다. 보이면 누를 수 있고, 가려져 있으면 누를 수 없습니다 — 별도의 표시가 필요 없도록 규칙과 시야를 일치시켰습니다.
완성	홀더에 같은 색 3개 가 모이면 즉시 비워지고 그 칸이 돌아옵니다.
낙하	판의 나사가 전부 빠지면 판이 떨어져 나가고 아래가 드러납니다.
결정	드러난 나사의 색은 바로 그 순간 정해집니다 . 회색으로 한 박자 머물렀다가 색이 입혀지는 것이 그 장면입니다.
종료	손이 닿는 나사가 전부 들어갈 홀더가 없는 색 이 되면 런이 끝납니다.

회색 나사는 뒷면이 아닙니다. 색이 아직 없는 나사입니다. 그래서 뒤집어 놓은 카드처럼 그리지 않고, 가공되지 않은 맨 금속으로 그립니다. 이 게임에서 가장 중요한 것이 화면에 그대로 드러나 있습니다 — 패널을 열지 않아도.

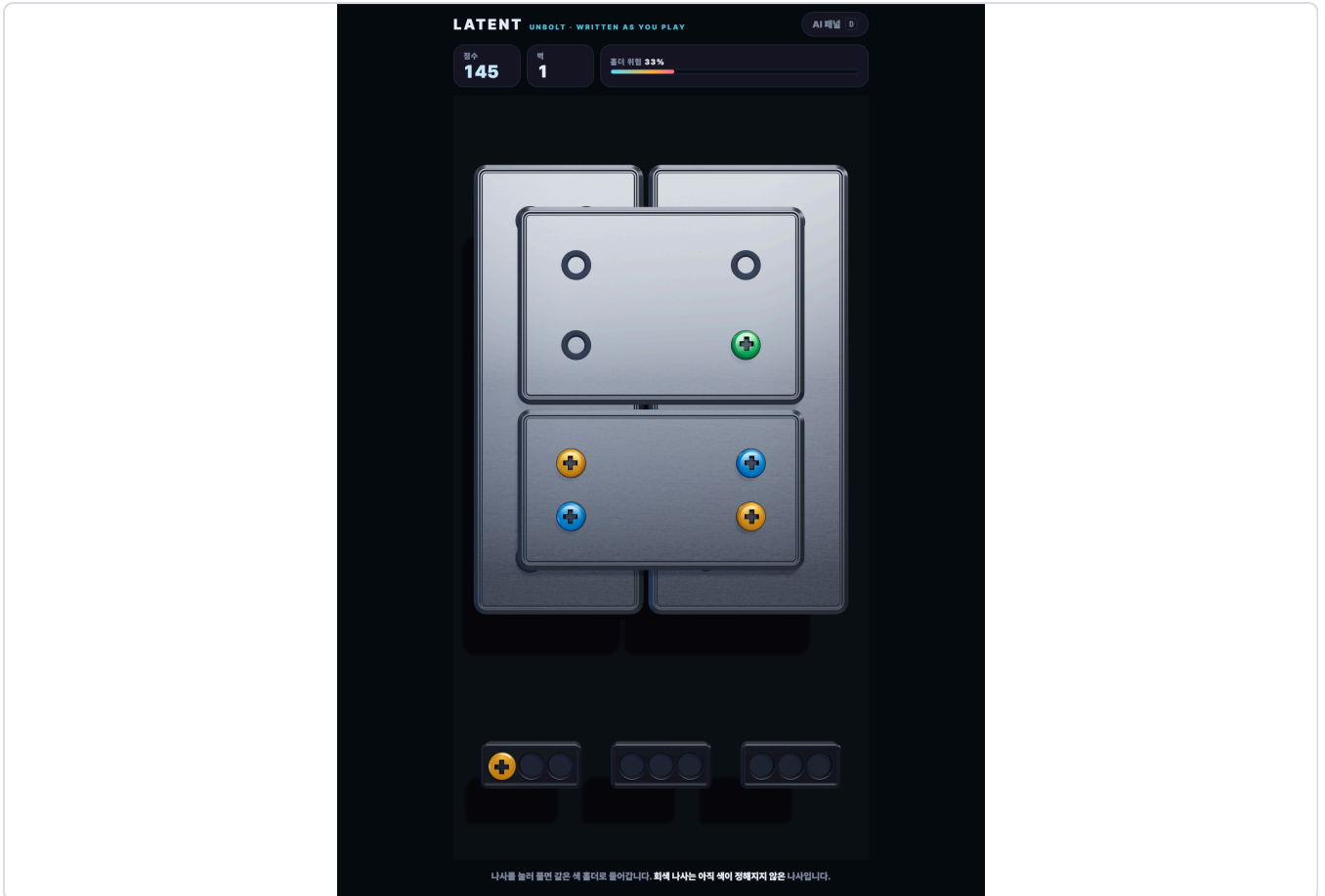
무엇이 실력인가

색이 즉석에서 정해진다면 플레이어가 할 일이 없어 보일 수 있지만, 반대입니다. 선택은 **어떤 나사를 어떤 순서로 풀 것인가**입니다.

- 이미 열린 홀더와 같은 색을 먼저 풀면 그 칸이 빨리 돌아옵니다
- 새 색을 덜컥 집으면 홀더 하나가 **세 개가 모일 때까지 묶입니다**
- 무엇보다 **끝낼 수 없는 색에 홀더를 묶지 않는 것** — 벽에 남은 같은 색이 하나뿐인데 홀더를 그 색으로 잠그면, 그 칸은 이 벽이 끝날 때까지 돌아오지 않습니다
- 어떤 나사를 빼면 판이 떨어져 아래가 여러 개 열립니다 — 선택지를 늘리는 수입니다

한 턴에 손이 닿는 나사는 평균 **4.3개**입니다. 열일곱 개 중에 고르는 것은 선택이 아니어서, 아래 판의 볼트 위치를 위 판이 실제로 덮도록 설계를 고쳤습니다. 그리고 시스템은 **매 턴마다** 그 선택을 솔버의 최선수와 비교해 채점합니다.

화면 구성



실제 플레이 화면. 철판이 겹으로 물려 있고, 손이 닿는 나사만 색을 가지고 있습니다. 나사가 빠진 자리에는 볼트 구멍이 남아, 그 판을 얼마나 해체했는지가 숫자 없이 읽힙니다. 아래가 홀더 3칸입니다. 화면의 나사·철판·구멍·홀더는 모두 Blender 스크립트가 생성한 3D 모델입니다.

AI 패널

우상단 “AI 패널” 버튼 또는 **D** 키로 열립니다. 이 게임의 AI가 무엇을 근거로 방금 그 색을 정했는지 숨기지 않고 전부 보여줍니다.

- 지금 시스템이 당신을 어떻게 추정하는지 — 숙련도 0와 계획성·정돈·템포, 좌절도·지루함
- 숙련도 · 목표 난이도 · 실제 난이도가 겹쳐 그려지는 **플로우 채널 그래프**
- 방금 정한 색의 근거 (한국어 문장)
- 검토된 후보 색들과 채택되지 않은 것들 — 완결성 판정 포함
- 불변식 검증: 색이 정해지지 않은 나사 수 $\geq$ 완성에 필요한 나사 수
- 패널이 열려 있는 동안 **AI가 방금 결정한 나사가 화면에서 강조됩니다**

기술적 세부 사항은 별도 제출물 ④ **AI 활용 기술 문서**에 정리했습니다.

실행 방법

플레이 (심사자용)

위 플레이 링크를 클릭하면 끝입니다. 설치 과정이 없습니다. 모바일 브라우저와 데스크톱 브라우저 모두에서 동작하며, 세로 화면을 기준으로 설계했습니다.

소스에서 직접 빌드

```
git clone https://github.com/TaraGamesDev/latent-nan2026.git
cd latent-nan2026
npm install

npm run dev      # 개발 서버 - http://localhost:5173
npm run build    # docs/ 로 정적 빌드 (GitHub Pages가 서빙하는 산출물)
npm run bench    # 헤드리스 엔진 검증 (페르소나당 300런)
npm run margin   # 이 게임에서 실력이 얼마나 값어치가 있는지 측정
```

Node.js 20 이상이면 동작합니다 (개발 환경은 Node 22.14). `npm run bench` 는 심사자가 직접 검증하는 데도 쓸 수 있습니다 — 봇 페르소나로 전체 게임을 실제 `tap()` 경로로 돌려 네 가지를 **단언**합니다: 완결성 불변식이 매 탭 유지되는가, 약한 플레이 어는 반드시 지는가, 실력이 실제로 값을 하는가, 숙련도 추정이 페르소나를 마땅한 순서로 정렬하는가.

제작 범위

개인 참가입니다. 기획·게임 디자인·엔진·AI 시스템·3D 렌더러·UI·문서 전부를 직접 작성했으며, 개발 과정에서 Anthropic Claude를 코딩 협업 도구로 사용했습니다. 사용 내역은 ④ **AI 활용 기술 문서**에 빠짐없이 기재했습니다.

구매·스톡 에셋은 사용하지 않았습니다. 화면의 나사·철판·볼트구멍·홀더는 **Blender 파이썬 스크립트가 생성한 3D 모델**이고, 그 스크립트는 저장소에 들어 있어 `npm run models` 로 같은 바이트가 다시 나옵니다. 금속 표면의 브러시드 러프니스 맵 1장은 Comfy Cloud에서 오픈 웨이트 모델로 생성한 뒤 자체 코드로 심리스 처리한 것이며, 생성 경로 전체를 `assets/textures/PROVENANCE.md` 에 기록했습니다. 사운드와 폰트는 사용하지 않았습니다(시스템 폰트).

런타임 의존성은 렌더링에 쓰는 Three.js 하나입니다. 전송량은 **JS 169KB · CSS 3KB · 모델 83KB · 텍스처 55KB, 합계 약 310KB**(gzip 기준)이고 외부 요청은 0건입니다.